

Programación

Bloque 03 - Arrays

Índice

1. Introducción
2. Arrays
 - 2.1. Declaración y creación de arrays
 - 2.2. Uso de arrays
 - 2.3. Recorrer arrays
 - 2.3.1. Ciclo for clásico
 - 2.3.2. Ciclo forEach
 - 2.4. Búsquedas en arrays
 - 2.5. Arrays multidimensionales
 - 2.6. Algoritmos de búsqueda
 - 2.7. Algoritmos de ordenación

1.- Introducción

Hasta este momento sólo hemos utilizado variables para almacenar datos. Aunque esto tiene su utilidad, en muchas ocasiones es necesario no almacenar datos "sueltos", sino conjuntos de datos. Para esta misión existe lo que se denominan **estructuras de información**, que son estructuras, como su nombre indica, que permiten almacenar un conjunto de datos.

Hay muchos tipos de estructuras debido a que hay muchas necesidades operativas distintas. Cada una de las estructuras presenta una serie de ventajas y una serie de inconvenientes y es labor del programador el balancear estas ventajas e inconvenientes y elegir la estructura más apropiada para la tarea a realizar.

En esta unidad veremos la estructura más básica, los arrays y, en temas posteriores, procederemos a desarrollar otras estructuras existentes.

Esto se va a hacer así porque esta estructura es ampliamente utilizada en otros lenguajes de programación y es necesario conocerlas bien para poder usarlos correctamente. Por ejemplo, en el módulo Lenguaje de Marcas se hace uso de JavaScript y es necesario saber utilizar los arrays para poder manejar el DOM.

2.- Arrays

Los arrays son la estructura de almacenamiento más básica, aunque ofrecen funcionalidad suficiente para muchas tareas. Sin embargo pecan de poco flexibles.

Un array es una estructura de datos que puede almacenar un conjunto de datos **del mismo tipo**. Podemos ver, en cierto modo, a un array como a un conjunto de variables del mismo tipo colocadas bajo un mismo nombre.

2.1.- Declaración y creación de arrays

Un array se declara de la siguiente forma:

```
tipo[] nombre;
```

donde `tipo` es el tipo de todos los elementos del array y `nombre` es el nombre del array. Nótese los paréntesis cuadrados.

Esto declara el array pero no lo crea, es decir, no tiene aún asignado espacio para almacenar datos. Además hay que decir que las variables de tipo array funcionan como las de tipo objeto: almacenan una referencia al array, no el array en sí. Cuando se accede al array a través de la variable se de referencia esta y se accede al array real. Esto significa que si asignamos un array a otro no se hace una copia del array sino que, al igual que ocurre con los objetos, las dos variables apuntarán **al mismo array**.

Para crear un array hay que utilizar, al igual que para los objetos, la instrucción `new`, de la forma:

```
nombre = new tipo[cantidad];
```

donde `nombre` es el nombre de una variable array del tipo adecuado, `tipo` es el tipo de los elementos del array y `cantidad` es la cantidad de elementos que va a almacenar el array.

Por ejemplo:

```
calificaciones = new double[10];
```

Crea un nuevo array de 10 elementos de tipo `double` y almacena la referencia en la variable `calificaciones`, que debe haber sido declarada de tipo `double[]`.

Es bastante usual hacer las dos cosas al mismo tiempo, declarar la variable para el array y crearlo, haciendo:

```
tipo[] nombre = new tipo[cantidad];
```

como por ejemplo:

```
double[] calificaciones = new double[10];
```

que crea la variable para almacenar la referencia al array y el array en sí en la misma línea.

Usando este método para crear arrays los elementos del mismo se inicializan a los valores por defecto típicos de Java, esto es, los números se inicializan a cero, las referencias a `null`, los `char`, carácter nulo y los booleanos a `false`.

Es posible, sin embargo, crear un array ya inicializado con valores distintos a estos, usando la siguiente sintaxis:

```
variable = {valor1, valor2, ....., valorn};
```

En este caso se crea el array con un tamaño igual al número de elementos entre llaves y cada elemento tiene inicialmente el valor que le corresponde por su posición en la lista de valores.

Ejemplo:

```
int[] edades = {25, 30, 45};
```

Crea el array `edades` con tres elementos, el primero contiene inicialmente 25, el segundo 30 y el tercero 45.

Es posible, aunque no muy útil tener un array de cero elementos, haciendo:

```
variable = new tipo[0];
```

o también

```
variable = {};
```

2.2.- Uso de arrays

Una vez declarado y creado, un array ya puede usarse. Como se ha comentado en secciones anteriores, los valores de los elementos pueden haber sido inicializados a valores concretos (usando las llaves) o a los valores por defecto de java (usando `new`).

Cada elemento de un array puede tratarse como si fuera una variable simple del tipo del array. Por lo tanto puede leerse de ella, empleándola en una expresión o puede modificarse asignándole un valor.

Para acceder a un elemento de un array se usa la sintaxis:

```
array[indice]
```

donde `array` es una variable con una referencia a un array e `indice` es un número entero positivo. El número `indice` es el que indica cual de los elementos del array estamos accediendo. Valdrá 0 para el primer elemento, 1 para el segundo y así sucesivamente. Si se utiliza un índice que no se corresponde con un elemento real del array se producirá una excepción no chequeada de tipo `java.lang.ArrayIndexOutOfBoundsException`

Siempre es posible obtener el número de elementos de un array empleando la expresión:

```
array.length
```

donde `array` debe ser una referencia a un array

```
int[] numeros = {10, 20, 30, 40};
System.out.println("El valor de la última posición del array es: " +
numeros[numeros.length - 1]);
```

```
System.out.println("El valor de la última posición del array es: " +
numeros[numeros.length - 1]);
```

Por lo tanto el rango de índices válidos para un array **cualquiera** es de 0 hasta `array.length - 1` Ejemplo:

```
int[] numeros = {10, 20, 30, 40};
System.out.println("El valor de la última posición del array es: " +
numeros[numeros.length - 1]);

numeros[numeros.length - 1] = 4000;

System.out.println("El valor de la última posición del array es: " +
numeros[numeros.length - 1]);
```

Debería imprimir por pantalla primero el valor 40 y después 4000

2.3.- Recorrer arrays

Una tarea muy frecuente es la de recorrer un array. Recorrer un array consiste en realizar una tarea que implica a todos los elementos de un array. Por ejemplo, supongamos un array

```
double[] calificaciones
```

que contiene las calificaciones de un alumno en un trimestre. Si queremos calcular la calificación media debemos emplear todas las calificaciones que hay en el array. Por tanto debemos recorrerlo. También podrás leerlo en algún sitio como iterar el array.

Para iterar un array hay dos métodos básicos. Aunque no estás limitado a ellos y podrías emplear otros métodos, estos son los más utilizados: Ciclo for clásico y ciclo `forEach`.

2.3.1.- Ciclo for clásico

En este método utilizamos un ciclo for clásico para recorrer el array. Empleamos un contador que va tomando los valores del índice desde 0 hasta el del último elemento del array (`array.length - 1`). Dentro del ciclo se emplea el índice para acceder al elemento en cuestión del array. Por ejemplo, para imprimir por pantalla todos los elementos de un array usando el ciclo for clásico, emplearíamos algo similar a lo siguiente:

```
// Creamos el array
double[] calificaciones = {7.5, 2.3, 8.2, 5.6, 9.2};

// Usamos un contador desde 0 hasta la longitud del array - 1
for (int i = 0; i < calificaciones.length; i++) {

    // Imprimimos la calificación
    System.out.println("La calificación " + i + "es de " +
        calificaciones[i]);
}
```

2.3.2.- Ciclo forEach

En un ciclo estilo forEach no vamos contando los elementos del array sino que empleamos directamente los valores del mismo.

La sintaxis básica es:

```
for (tipo variable: array) {
    // Usa variable para lo que se necesite
}
```

En el ciclo forEach se va asignando en cada iteración del ciclo un valor de array a la variable variable. Ésta debe ser del mismo tipo que los elementos del array, por supuesto, o un tipo compatible (número de menor precisión u objeto de subclase).

El mismo ejemplo de la sección anterior se haría usando forEach como se ve a continuación:

```
// Creamos el array
double[] calificaciones = {7.5, 2.3, 8.2, 5.6, 9.2};

// Usamos un ciclo forEach para recorrer el array
for (double calificacion: calificaciones) {
    // Imprimimos la calificación
    System.out.println("La calificacion es de " + calificacion);
}
```

Aparentemente son equivalentes, sin embargo hay algunas diferencias fundamentales:

En primer lugar no conocemos la posición dentro del array del elemento que estamos procesando en cada iteración. Esto provoca, por ejemplo, que en nuestro ejemplo ya no

digamos que elemento es cada uno. Podríamos haberlo hecho añadiendo un contador pero se ha omitido para resaltar el hecho de su falta.

En segundo lugar, no podemos modificar los elementos del array directamente. Esto es así porque en la variable se nos va proporcionando **una copia** de cada elemento del array, no el elemento en sí, por lo que no podemos modificarlo.

Por lo tanto, si queremos más control sobre el array deberíamos usar un ciclo clásico. Si simplemente queremos consultar los valores podemos usar el ciclo forEach.

2.4.- Búsquedas en arrays

Una operación muy común es la de realizar búsquedas en arrays para determinar, o bien si un determinado valor se encuentra entre los valores contenidos en el array (búsqueda), o bien no sólo si se encuentra o no sino también en qué posición dentro del array se encuentra, en caso de localizarlo (localización).

Para las búsquedas podemos usar tanto el ciclo for clásico como el forEach, aunque es más simple el uso de este último.

Un código que busca un entero determinado en un array de enteros sería como se ve a continuación:

```
// Para cada elemento
for(int elemento: array) {
    // Si es el buscado
    if (elemento == buscado) {
        // Terminamos con éxito
        System.out.println("Éxito");
    }
}
```

Si queremos localizar el elemento, la mejor opción es el for clásico:

```
private static int localizarArrayEntero(int[] array, int buscado) {
    // Para cada índice del array
    for (int i = 0; i < array.length; i++) {
        // Si el elemento en esa posición es el buscado
        if (array[i] == buscado) {
            // Termina y devuelve la posición return i;
        }
    }
    // Si se llega al final del ciclo es que no se ha encontrado
    // Devolvemos un valor establecido no válido en un array, por
    // ejemplo -1
    return -1;
}
```

```
}
```

2.5.- Arrays multidimensionales

Los elementos de un array pueden ser de cualquier tipo: primitivos, como `int` o `double`, objetos como `String`, **o incluso otros arrays**. En este caso tendríamos arrays con más de una dimensión, lo que se denominan **arrays multidimensionales**. En este caso el proceso es evidentemente más simple porque tenemos que movernos en más de una dimensión y en algunos casos es difícil el visualizar la forma de recorrerlos o buscar en ellos.

Para crear un array de dos dimensiones se haría de la forma:

```
tipo[][] nombre = new tipo[cantidadfilas][cantidadcolumnas]
```

donde `tipo` es el tipo de los elementos del array, `nombre` es el nombre del array, `cantidadfilas` es el número de filas que tiene el array y `cantidadcolumnas` es el número de columnas que tiene el array.

Podemos pensar en un array de dos dimensiones (bidimensional) como una tabla. La primera coordenada es la de la fila dentro de la que está el elemento y la segunda sería la columna que da la posición dentro de la fila en la que está efectivamente el elemento.

Para acceder a un elemento de un array bidimensional hay que emplear dos índices, de la forma:

```
array[filas][columnas]
```

Al igual que con los unidimensionales, hay que tener cuidado con que los índices no sobrepasen los valores permitidos u obtendremos la misma excepción.

Un array bidimensional Java lo ve como un array de arrays. El primer array contendría las filas que es cada una de ellas un array de elementos.

Por lo tanto también podemos acceder usando un sólo índice:

```
array[filas]
```

Esto devuelve el array correspondiente a la fila en cuestión, que se puede recorrer como un array o asignarlo a una variable.

Como Java ve un array bidimensional como un array de arrays, a nadie extrañará la forma de inicializar un array en dos dimensiones. Se hace anidando llaves:

Ejemplo:

```
int[][] numeros = {{1,2,3},{4,5,6},{7,8,9},{10,11,12}};
```

Esto crea un array bidimensional de 4 filas y 3 columnas. La primera fila contiene los números 1, 2 y 3, la segunda 4, 5 y 6, la tercera, 7, 8 y 9, y la cuarta 10, 11 y 12.

Ejemplo:

```
// Creamos el array bidimensional
// Con las tablas de multiplicar del 1 al 4 (filas 0 a 3)
// Cada fila es una tabla y cada columna contiene
// el múltiplo correspondiente a la fila * (columna + 1)

int[][] tablasMultiplicar = {
    {1,2,3,4,5,6,7,8,9},
    {2,4,6,8,10,12,14,16,18,20},
    {3,6,9,12,15,18,21,24,27,30},
    {4,8,12,16,20,24,28,32,36,40}
};

// Imprimimos cuánto vale 3 * 4
// El número de fila y columna es uno menos porque los índices
// comienzan por cero en lugar de por 1
System.out.println("3 * 4 vale " + tablasMultiplicar[2][3]);

// Podemos pasar una sola tabla e imprimirla
imprimeTabla(3, tablasMultiplicar[2]);

// Para recorrer la tabla se emplean dos ciclos
// Uno para recorrer las filas y otro para recorrer
// las casillas de una fila (las columnas)
for (int i = 0; i < tablasMultiplicar.length; i++) {
    System.out.println("Tabla del " + (i + 1));
    for (int j = 0; j < tablasMultiplicar[i].length; j++) {
        System.out.println((i + 1) + " * " + (j + 1) + " = " +
            tablasMultiplicar[i][j]);
    }
}

// Metodo alternativo usando forEach
for (int[] fila: tablasMultiplicar) {
    System.out.println("Nueva fila");
    for(int casilla: fila) {
        System.out.println("Elemento: " + casilla);
    }
}
```

En este ejemplo creamos e inicializamos un array bidimensional correspondiente a las tablas de multiplicar de los números 1 al 4. La tabla contiene 4 filas (correspondientes a los números 1 a 4) y 10 columnas (los múltiplos del número de la fila).

A continuación imprimimos un número cualquiera para ver cómo se accede a un elemento del array.

Después hacemos una demostración de cómo se puede pasar una fila como un array llamando al método privado `imprimeTabla` (intenta hacerlo tu) y pasándole una fila (`tablasMultiplicar[2]`). `imprimeTabla` imprime la fila que se le pasa (que es un array unidimensional)

Por último hacemos una demostración de cómo se puede recorrer la tabla completa empleando el `for` clásico y el método `forEach`. Como el array es bidimensional

necesitaremos dos ciclos. El primero va recorriendo las filas y el segundo va recorriendo el interior de cada fila. El interior se ejecuta completamente tantas veces como filas haya.

En el caso del método `forEach` hay que fijarse en el tipo de la variable en el ciclo más externo. En lugar de `int` (que es el tipo de la casilla) se emplea `int[]` porque lo que se obtiene en cada iteración es una fila, que es un array de `int`. En el ciclo interior se itera sobre esta variable, que es un array usando, ahora si, una de tipo `int`.

De la misma forma que tenemos arrays bidimensionales podríamos tener tridimensionales y de más dimensiones.

La forma de tratarlos es la misma pero teniendo en cuenta las múltiples dimensiones: Cada elemento de cada nivel es un array de nivel inferior. Los índices deben estar controlados para evitar "salirse" del array y hay que utilizar tantos ciclos como dimensiones para recorrer completamente el array.

A continuación tenemos un ejemplo de manejo de array tridimensional en Java

```
/*
Creamos un array tridimensional. Va a contener la cantidad de lluvia
en cada capital de Andalucía en un día determinado
La primera coordenada es la ciudad siendo 0 = Almeria, 1 = Cadiz,
etc. en orden alfabético
La segunda es el mes 0 = enero, 1 = febrero, etc.
La tercera es el día. Usaremos 31 para que valga para todos los
meses, aunque en algunos meses haya casillas vacías que no se emplean
Como es un ejemplo vamos a crear 2 capitales y 2 meses para no
alargarlo mucho
*/

double[][][] pluviometriaAndalucia = {
{
{13.15,10.96,5.98,3.27,3.04,0.50,15.18,0.26,19.74,3.18,12.72,13.31,6.
17,12.70,0.69,1.32,1.42,12.36,16.72,9.78,1.69,14.09,8.76,12.41,13.25,
6.79,15.41,12.39,18.80,5.47,8.91},
{11.35,16.34,6.01,17.76,13.92,18.26,5.10,6.15,19.24,10.94,1.94,16.69,
10.42,11.72,13.08,12.75,14.04,3.62,2.59,14.70,9.17,0.48,0.84,19.45,11
.21,19.65,10.86,1.11,14.79,14.90,13.48
}
},
{
{9.62,8.21,4.90,7.68,11.66,17.42,15.95,19.33,5.37,14.40,10.51,9.39,11
.56,14.39,16.80,9.4
2,2.56,17.69,5.37,9.25,9.21,8.91,7.55,7.59,18.90,2.80,5.78,15.20,13.2
8,19.72,9.82},
{10.97,15.53,18.82,9.25,3.73,12.64,16.35,6.55,11.63,15.80,6.80,1.24,1
8.52,15.56,9.01,1.3
6,2.48,0.48,9.64,4.64,16.54,15.96,13.44,5.05,7.37,17.23,19.43,12.17,1
1.24,8.36,16.37}
}
}
```

```

}
};

// Imprimimos cuánto llovió en Cádiz el 10 de Febrero
// El número de tabla, fila y columna es uno menos porque los índices
// comienzan por cero en lugar de por 1

System.out.println("El 10 de Febrero cayeron en Cádiz " +
pluviometriaAndalucia[1][1][9] + " litros por metro cuadrado");

// Podemos pasar una sola tabla e imprimirla
imprimeTabla(1, pluviometriaAndalucia[0]);

// o sólo un mes
imprimeMes(1,1, pluviometriaAndalucia[0][0]);

/*
Para recorrer el array se emplean tres ciclos.
El primero recorre las tablas, el segundo las filas y el tercero las
columnas
*/

for (int i = 0; i < pluviometriaAndalucia.length; i++) {
    System.out.println("Pluviometria de la capital " + (i + 1));
    for (int j = 0; j < pluviometriaAndalucia[i].length; j++) {
        System.out.println("Datos del mes " + (j + 1));
        for (int k = 0; k <
pluviometriaAndalucia[i][j].length; k++)
            { System.out.println("El dia " + (k + 1) + " se
recogieron " + pluviometriaAndalucia[i][j][k] + " litros por metro
cuadrado");
            }
        }
    }

// Metodo alternativo usando forEach
for (double[][] tabla: pluviometriaAndalucia) {
    System.out.println("Nueva tabla");
    for (double[] fila: tabla) {
        System.out.println("Nueva fila");
        for(double casilla: fila) {
            System.out.println("Elemento: " + casilla);
        }
    }
}

```

En primer lugar se crea el array. Fíjate en cómo se crea una estructura anidada de tres niveles. El primer nivel corresponde a la ciudad y define internamente varias tablas. El segundo corresponde al mes y define internamente varios arrays unidimensionales. El tercero es la lluvia de cada día correspondiente a esa capital y mes.

A continuación se muestra como acceder a un dato en concreto dentro del array.

También se muestra como pasar parte del array. En este caso se pueden pasar tablas completas (los datos de una capital) o arrays unidimensionales (los datos de un mes).

También se muestra cómo recorrer completamente los arrays empleando ambos métodos (for- clásico y forEach).

2.6 Algoritmos de búsqueda

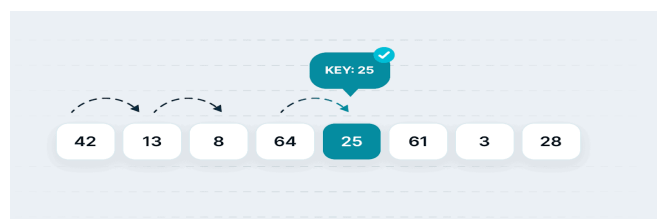
Los algoritmos de búsqueda te ayudan a encontrar lo que buscas en grandes colecciones de datos. Estos algoritmos utilizan reglas y procesos sistemáticos para utilizar el menor número posible de comparaciones o iteraciones para encontrar los elementos de búsqueda deseados.

Estos algoritmos pueden utilizarse para cualquier conjunto de datos. Nosotros por comodidad vamos a verlos como ejemplo en esta unidad, aunque pueden utilizarse en otras estructuras.

Es importante conocer estos algoritmos ya que en cualquier momento nos pueden hacer falta en el mundo de la programación y, según su nos convenga, podemos utilizar uno u otro. Podemos distinguir los siguientes algoritmos:

2.6.1. Búsqueda lineal o secuencial

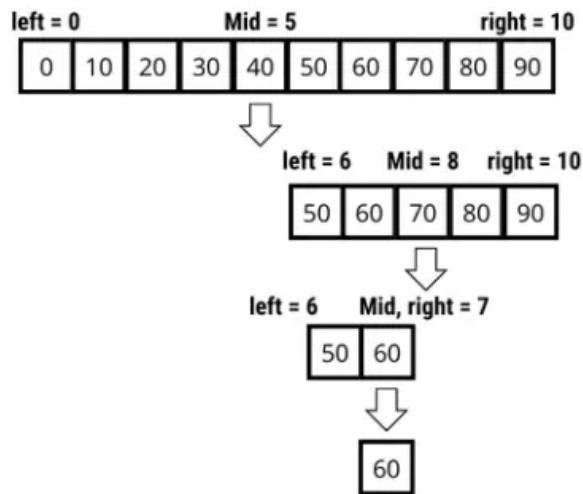
Este método, el más básico, recorre el array desde el primero hasta el último elemento. En el caso de encontrar una coincidencia, no recorre el resto del array y devuelve “encontrado”. Opcionalmente también podría devolver la posición del array donde ha encontrado la coincidencia. Se suele utilizar cuando no hay gran cantidad de datos o la estructura de datos está desordenada.



2.6.1. Búsqueda binaria

Divide repetidamente el array a la mitad hasta que encuentra el elemento o determina que no está presente.

Target = 60



2.7 Algoritmos de ordenación

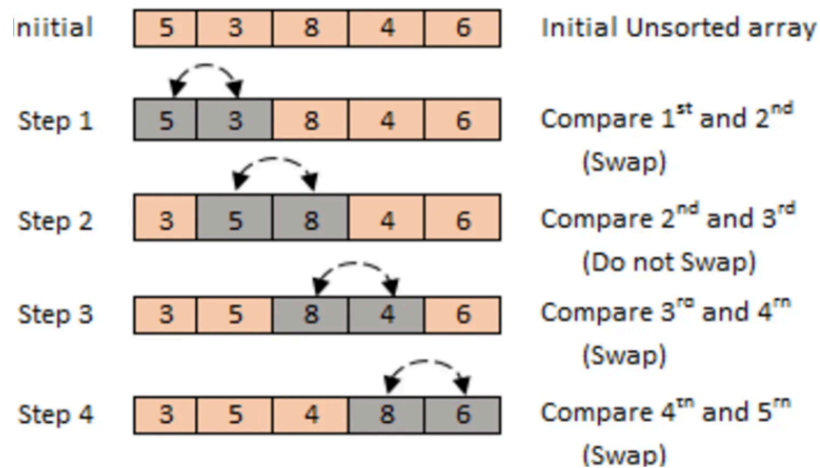
Los algoritmos de ordenación nos permiten ordenar los datos presentes en una colección de datos. Para realizar una búsqueda y tener una respuesta lo más rápida posible, nos interesa que la estructura de datos esté ordenada, ya que en caso contrario habrá que ir recorriendo la estructura hasta encontrar el elemento buscado. Esta ordenación requiere tiempo, y es lo que tendremos que intentar que sea lo menor posible.

Hay gran cantidad de algoritmos de ordenación. En este curso vamos a explicar tres de ellos para tener una visión de las posibilidades que podemos tener.

2.7.1 Algoritmo de la burbuja

Recorre el array comenzando por el primer elemento, comprobará que esté ordenado respecto al siguiente, si está ordenado, pasará al segundo elemento, sino, los intercambiará y después pasará al segundo elemento. Este proceso se repite hasta que se llegue al último elemento y volvamos a empezar.

El proceso acaba cuando en una pasada no ha habido elementos que intercambiar.

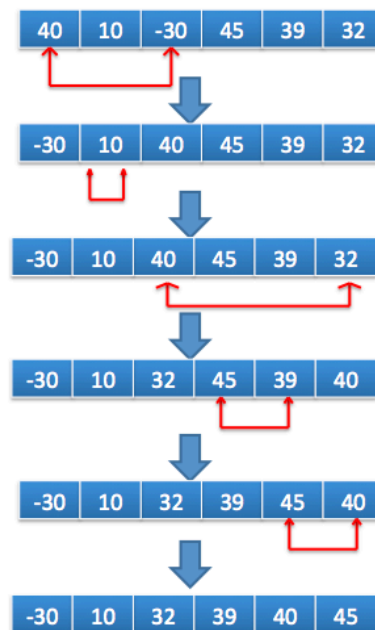


2.7.2 Ordenación de selección

Este método junto al de ordenación de la burbuja es de los más simples que nos podemos encontrar, pero no son los más eficientes. Es útil cuando no hay gran cantidad de datos.

El proceso es el siguiente:

1. Encuentra el elemento más pequeño del array y lo intercambia con el primer elemento.
2. Encuentra el segundo elemento más pequeño y lo intercambia con el segundo elemento del array.
3. Encuentra el tercer elemento más pequeño y lo intercambia con el tercer elemento del array.
4. Se repite este proceso hasta que se complete el array.



Selection sort

2.7.3 Ordenación por Quick-sort

Quick sort es un algoritmo muy eficiente de ordenación siguiendo la filosofía del “divide y vencerás”. La idea es dividir el array inicial en dos subarrays cada vez y ordenar estos elementos arrays más pequeños. Los elementos menores se colocarán a la derecha y los mayores a la izquierda.

Este método de ordenación es un claro ejemplo de programación recursiva.

Los pasos involucrados en Quick-sort son:

1. Elige un elemento para que sirva como pivote, puede ser el primer elemento, el último o uno al azar.
2. Ordena el array de forma que todos los elementos menores que el pivote estén a la izquierda y todos los elementos mayores que el pivote estén a la derecha.
3. Llama a Quicksort de forma recursiva, teniendo en cuenta el pivote anterior para subdividir adecuadamente los arreglos izquierdo y derecho.
4. El proceso acabará cuando todos los subarrays tengan dos o menos elementos, ya que todos estarán ordenados.

